Prelab 7.3: Two AVRs Connected Serially via SPI

Objective: Replace UART with a full-duplex SPI communication system. Two identical AVR systems exchange switch states (PC0/PC1) to control remote LEDs (PB0/PB1) using non-blocking interrupt-driven logic.

---

1. Master System (System 1)

1.1 Initialization Registers

Register	Value	Configuration Logic	Code (C Language)
DDRB	0x2F	PB0,1 (LEDs), PB2 (SS), PB3 (MOSI), PB5 (SCK) = Output; PB4 (MISO) = Input	DDRB = (1 << PB0) | (1 << PB1) | (1 << PB2) | (1 << PB3) | (1 << PB5);
DDRC	0x00	PC0, PC1 = Input (Switches)	DDRC &= ~((1 << PC0) | (1 << PC1));
PORTC	0x03	Enable internal pull-ups for PC0, PC1	PORTC |= (1 << PC0) | (1 << PC1);
PORTB	0x04	Initialize SS (PB2) High (Idle)	PORTB |= (1 << PB2);
SPCR	0xD0	Enable SPI, Enable Interrupt, Master Mode, f/4 Clock	SPCR = (1 << SPIE) | (1 << SPE) | (1 << MSTR);

TCCR0A	0x02	Timer0: CTC Mode	TCCR0A = (1 << WGM01);
TCCR0B	0x05	Timer0: Prescaler 1024 (~10ms period @ 16MHz)	TCCR0B = (1 << CS02) | (1 << CS00);
OCR0A	155	Compare value for ~100Hz frequency	OCR0A = 155;
TIMSK0	0x02	Enable Timer0 Compare Match A Interrupt	TIMSK0 = (1 << OCIE0A);

1.2 Master Operational Logic

Action	Register/Bit	Purpose
Input	PINC (PC0, PC1)	Read local switch states.
Control	PORTB (PB2)	Pull SS Low to initiate transfer; High when finished.
Data	SPDR	Write to start transmission; Read to retrieve Slave data.
Output	PORTB (PB0, PB1)	Update local LEDs based on received data.

---

2. Slave System (System 2)

2.1 Initialization Registers

Register	Value	Configuration Logic	Code (C Language)

DDRB	0x13	PB0,1 (LEDs), PB4 (MISO) = Output; PB2 (SS), PB3 (MOSI), PB5 (SCK) = Input	DDRB = (1 << PB0) | (1 << PB1) | (1 << PB4);
DDRC	0x00	PC0, PC1 = Input (Switches)	DDRC &= ~((1 << PC0) | (1 << PC1));
PORTC	0x03	Enable internal pull-ups for PC0, PC1	PORTC |= (1 << PC0) | (1 << PC1);
SPCR	0xC0	Enable SPI, Enable Interrupt, Slave Mode	SPCR = (1 << SPIE) | (1 << SPE);
SPDR	Initial	Preload switch state for the first exchange	SPDR = initial_sw_state;

2.2 Slave Operational Logic

Action	Register/Bit	Purpose
Data	SPDR	Read received Master data; Write to preload next response.
Output	PORTB (PB0, PB1)	Update local LEDs based on received Master data.
Input	PINC (PC0, PC1)	Sample switches to prepare for the next clock cycle.

---

3. Program Flowcharts

Master Flow (Timer-Driven)

1. **Timer ISR (~10ms):**
   - Check if spi_busy flag is clear.
   - Read PINC, invert bits (active low switches), and pull **SS Low**.
   - Load SPDR with data; set spi_busy = 1.
2. **SPI ISR:**
   - Read SPDR (data from Slave).
   - Update PORTB LEDs.
   - Pull **SS High**.
   - Set spi_busy = 0.

Slave Flow (Event-Driven)

1. **Start:** Preload SPDR with current switch state.
2. **SPI ISR (Triggered by Master Clock):**
   - Read SPDR (data from Master).
   - Update PORTB LEDs.
   - Sample PINC and load SPDR immediately to prepare for the *next* exchange.